```
    k=3;
    std::cout<<f()<<std::endl;
}
```

This final variant allows you to ensure that only the intended variables are being captured, because any reference to a local variable not in the capture list will cause a compilation error. If you choose this option, you have to be careful when accessing class members if the function containing the lambda is a member function. Class members can't be captured directly; if you wish to access class members from your lambda, you have to capture the `this` pointer by adding it to the capture list. In the following example, the lambda captures `this` to allow access to the `some_data` class member:

```
struct X
{
    int some_data;
    void foo(std::vector<int>& vec)
    {
        std::for_each(vec.begin(),vec.end(),
            [this](int& i){i+=some_data;});
    }
};
```

In the context of concurrency, lambdas are most useful as predicates for `std::condition_variable::wait()` (section 4.1.1) and with `std::packaged_task<>` (section 4.2.1) or thread pools for packaging small tasks. They can also be passed to the `std::thread` constructor as a thread function (section 2.1.1) and as the function when using parallel algorithms such as `parallel_for_each()` (from section 8.5.1).

A.6. Variadic templates

Variadic templates are templates with a variable number of parameters. Just as you've always been able to have variadic functions such as `printf` that take a variable number of parameters, you can now have variadic templates that have a variable number of *template* parameters. Variadic templates are used throughout the C++ Thread Library. For example, the `std::thread` constructor for starting a thread (section 2.1.1) is a variadic function template, and `std::packaged_task<>` (section 4.2.2) is a variadic class template. From a user's point of view, it's enough to know that the template